07 快速排序 解答

07-E01 Lomuto 分割追蹤

推導

從 j=1 起掃描。7>2 不交換；3>2；8>2；3>2；6>2，均不交換。j=6 是 pivot 本身不掃，最後交換位置 1 與 pivot 位置 6，得到 [2,3,8,3,6,7]，pivot 最終索引 1。

結論與檢核

左側空、右側 [3,8,3,6,7]。此組資料顯示 pivot 為最小時最不平衡；同一資料若取中位附近可得到較好分割。

07-E02 1:9 分割的遞迴界

推導

每層的問題成本總和不超過 n，最長分支每層乘 0.9，到規模 1 需 O(log n) 層。其他分支在相同或更淺深度結束，故全樹總成本不超過常數乘 n log n。

結論與檢核

因此總成本為O(n log n)。此論證只需每個子問題的最大子規模不超過其父規模的0.9倍，無須每次恰為1:9。

07-E03 四個元素端點被比較的機率

推導

元素 1 與 4 不被某一側分開的唯一可能在區間 [1,2,3,4] 中先抽到 1 或 4。首個被抽中的 pivot 均等為四個位置之一，因此兩者比較機率為 2/4=1/2。

結論與檢核

若先抽到 2，則 1 在左、4 在右，永不再同區間；若先抽到 3 同理。中間元素先成為 pivot 會隔離兩端，這是比較機率不是 1 的原因。

07-E04 全相等輸入與隨機Pivot

推導

任何選中的pivot值都5，迴圈每個<=測試成立，最後q=r。剩左側n−1鍵，故C(n)=C(n−1)+(n−1)、C(0)=C(1)=0，解為n(n−1)/2。隨機選擇不改變任何分割規模，所以其期望也相同。

07 快速排序 解答 （續）

結論與檢核

二路版本的期望O(n log n)分析需相異鍵前提。改三路partition，得到<5、=5、>5三區，此例等值區含全部n鍵，兩側空，不再遞迴，只花Θ(n)。一般情況遞迴兩個嚴格區，不能再重複處理等值區。

07-E05 只遞迴較小側的堆疊界

推導

partition後兩側長度a,b滿足a+b=n−1，較小側min(a,b)<=floor((n−1)/2)。沿真正遞迴的frame鏈，規模至少減半，故至多O(log(n+1))層；while處理較大側不額外堆疊frame。

結論與檢核

每次pivot最大時，較小側空而while繼續n−1側，stack甚至只需常數層；但partition比較依序n−1,n−2,⋯,1，總Θ(n^2)。空間界與時間界是不同成本，減少遞迴frame沒有消除分割工作。

07-E06 含負數與重複值的分割不變量

推導

初i0。j1的−1<=2，i1、自交換，陣列不變；j2的4>2，i仍1；j3的0<=2，i2、交換2與3，得[−1,0,4,4,2]；j4的4>2，i仍2。最後交換i+1=3與r5，得[−1,0,2,4,4]，q3。

結論與檢核

左[−1,0]<=2，右[4,4]>2，partition只保證區域間大小關係。本例兩側剛好有序，不能推論一般情況都如此；例如正文[9,3,7,1,8,2,6]得到左[3,1,2]、右[8,7,9]，仍須遞迴排序。

07-E07 區間外 pivot、比較事件與計數模型

由第一個區間內 pivot 分情形

首先選中的區間鍵	本次是否比較 2、5	後續是否可能補比較
2	是，2 是 pivot	不需要；2 已退出遞迴
3	否，2、5 分到不同側	否
4	否，2、5 分到不同側	否
5	是，5 是 pivot	不需要；5 已退出遞迴

07 快速排序 解答 （續）

每個區間鍵等可能最先被選中，所以機率為 2/4=1/2。區間外 1、6、7 先成為 pivot 時，整個 {2,3,4,5} 留在同一個子問題；它們不決定這一對是否比較。因此不能用 2/7。

條件事件

首個 pivot=7 時，2、5 一起留在 {1,⋯,6}，未來仍條件均勻抽樣；第一個區間鍵是端點的機率仍為 1/2。首個 pivot=3 時，2 已在左、5 已在右，且本次只把其他鍵與 3 比較，故所求條件機率為 0。

精確計數與邊界

本章每個非 pivot 鍵只做一次 `<= pivot` 鍵比較；索引測試不計入。三路版的一次分類可執行 `<`、`>` 兩次測試，即使分類次數仍是線性，也不能把同一精確數字稱為布林比較總數。互異鍵與均勻 pivot 的前提下，常數倍比較仍保留期望 Θ(n log n)，但精確常數須依實作重算。

本章 A[p..r] 是 1 起算閉區間，排除 pivot q 後遞迴 A[p..q−1]、A[q+1..r]。資料結構第 7 章 A[l:r] 是 0 起算、右端不含，三路回傳等值區 [lt,gt)，只遞迴 A[l:lt]、A[gt:r]。請依各章明定的契約推導索引，不要只照抄相似的函式名稱。

原創驗算程式附錄: partition_trace.py

```
"""Trace Lomuto partition on the course example."""

def partition(a):
    pivot = a[-1]
    i = -1
    for j in range(len(a) - 1):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[-1] = a[-1], a[i + 1]
    return i + 1


sample = [7, 3, 8, 3, 6, 2]
pivot_index = partition(sample)
assert all(x <= sample[pivot_index] for x in sample[:pivot_index])
assert all(x > sample[pivot_index] for x in sample[pivot_index + 1:])
print(sample, pivot_index)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

07 快速排序 解答 （續）

原創驗算程式附錄: quicksort_01_core.py

```
"""Lomuto quicksort with optional smaller-side recursion."""

def sort(values,small_stack=False):
    a=list(values)
    comparisons=[]
    deepest=0
    def partition(p,r):
        pivot=a[r]
        i=p-1
        for j in range(p,r):
            comparisons.append((a[j],pivot))
            if a[j]<=pivot:
                i+=1
                a[i],a[j]=a[j],a[i]
        a[i+1],a[r]=a[r],a[i+1]
        return i+1
    def run(p,r,depth=1):
        nonlocal deepest
        deepest=max(deepest,depth)
        while p<r:
            q=partition(p,r)
            if not small_stack:
                run(p,q-1,depth+1)
                run(q+1,r,depth+1)
                return
            if q-p<r-q:
                run(p,q-1,depth+1)
                p=q+1
            else:
                run(q+1,r,depth+1)
                r=q-1
    run(0,len(a)-1)
    return a,comparisons,deepest
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

07 快速排序 解答 （續）

原創驗算程式附錄: quicksort_checks.py

```
"""Finite checks for the companion quicksort core."""
from fractions import Fraction
from itertools import permutations, product
from math import factorial, log2, ceil
from quicksort_01_core import sort

def main():
    cases=0
    for length in range(8):
        for values in product(range(3),repeat=length):
            for reduced_stack in (False,True):
                result,_,depth=sort(values,reduced_stack)
                assert result==sorted(values)
                if reduced_stack:
                    assert depth<=ceil(log2(length+1))+1
            cases+=1
    for n in range(2,8):
        total=0
        pair_counts={}
        for values in permutations(range(1,n+1)):
            _,comp,_=sort(values)
            total+=len(comp)
            for x,y in comp:
                pair=tuple(sorted((x,y)))
                pair_counts[pair]=pair_counts.get(pair,0)+1
        h=sum((Fraction(1,k) for k in range(1,n+1)),Fraction())
        assert Fraction(total,factorial(n))==2*(n+1)*h-4*n
        for (i,j),count in pair_counts.items():
            assert Fraction(count,factorial(n))==Fraction(2,j-i+1)
    for n in (4,10,64):
        _,comparisons,depth=sort([5]*n,True)
        assert len(comparisons)==n*(n-1)//2 and depth<=2
    print(f"PASS arrays={cases}; exact means and pair probabilities n=2..7")
    print("PASS all-equal comparison count and smaller-side stack")


if __name__=="__main__":
    main()
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study